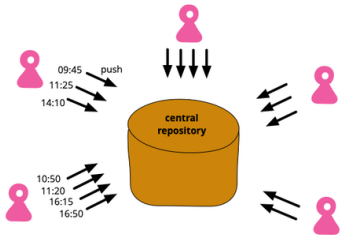
Continuous Integration

Continuous Integration is a software development practice where each member of a team merges their changes into a codebase together with their colleagues changes at least daily. Each of these integrations is verified by an automated build (including test) to detect integration errors as quickly as possible. Teams find that this approach reduces the risk of delivery delays, reduces the effort of integration, and enables practices that foster a healthy codebase for rapid enhancement with new features.



18 January 2024

Martin Fowler

AGILE

CONTINUOUS DELIVERY

EXTREME PROGRAMMING

CONTENTS

Building a Feature with Continuous Integration
Practices of Continuous Integration
- Put everything in a version controlled mainline
- Automate the Build
- Make the Build Self-Testing
- Everyone Pushes Commits To the Mainline Every Day
- Every Push to Mainline Should Trigger a Build
- Fix Broken Builds Immediately
- Keep the Build Fast
- Hide Work-in-Progress
- Test in a Clone of the Production Environment
- Everyone can see what's happening
- Automate Deployment

Styles of Integration
Benefits of Continuous Integration
- Reduced risk of delivery delays
- Less time wasted in integration
- Less Bugs

Table of Contents

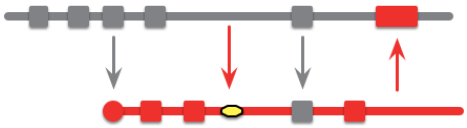
Enables Refactoring for sustained productivity
Release to Production is a business decision
When we should not use Continuous Integration
Introducing Continuous Integration
Common Questions
Where did Continuous Integration come from?
What is the difference between Continuous Integration and Trunk-Based Development?
Can we run a CI Service on our feature branches?
Can a team do both Continuous Integration and Feature Branching at the same time?
What is the difference between Continuous Integration and Continuous Delivery?
How does Continuous Deployment fit in with all this?
How do we do pull requests and code reviews?
How do we handle databases?
Final Thoughts

SIDEBARS

Semi-Integration (it isn't Continuous Integration)

I vividly remember one of my first sightings of a large software project. I was taking a summer internship at a large English electronics company. My manager, part of the QA group, gave me a tour of a site and we entered a huge, depressing, windowless warehouse full of people working in cubicles. I was told that these programmers had been writing code for this software for a couple of years, and while they were done programming, their separate units were now being integrated together, and they had been integrating for several months. My guide told me that nobody really knew how long it would take to finish integrating. From this I learned a common story of software projects: integrating the work of multiple developers is a long and unpredictable process.

Patterns for Managing Source Code Branches



This article explores Continuous Integration from another perspective, that of considering different strategies to handling branching. It goes into the purpose of branching, what it means to properly integrate, and the value of high-frequency integration. It's not so much a how-to on Continuous Integration, and doesn't talk about the practices to do it well. Rather it looks at the wider context of Continuous Integration with various other branching patterns.

by Martin Fowler ARTICLE
28 May 2020

Read more...

CONTINUOUS DELIVERY COLLABORATION VERSION CONTROL

I haven't heard of a team trapped in such a long integration like this for many years, but that doesn't mean that integration is a painless process. A developer may have been working for several days on a new feature, regularly pulling changes from a common main branch into her feature branch. Just before she's ready to push her changes, a big change lands on main, one that alters some code that she's interacting with. She has to change from finishing off her feature to figuring out how to integrate her work with this change, which while better for her colleague, doesn't work so well for her. Hopefully the complexities of the change will be in merging the source code, not an insidious fault that only shows when she runs the application, forcing her to debug unfamiliar code.

At least in that scenario, she gets to find out before she submits her pull request. Pull requests can be fraught enough while waiting for someone to review a change. The review can take time, forcing her to context-switch from her next feature. A difficult integration during that period can be very disconcerting, dragging out the review

process even longer. And that may not even the be the end of story, since integration tests are often only run after the pull request is merged.

In time, this team may learn that making significant changes to core code causes this kind of problem, and thus stops doing it. But that, by preventing regular refactoring, ends up allowing cruft to grow throughout the codebase. Folks who encounter a crufty code base wonder how it got into such a state, and often the answer lies in an integration process with so much friction that it discourages people from removing that cruft.

But this needn't be the way. Most projects done by my colleagues at Thoughtworks, and by many others around the world, treat integration as a non-event. Any individual developer's work is only a few hours away from a shared project state and can be integrated back into that state in minutes. Any integration errors are found rapidly and can be fixed rapidly.

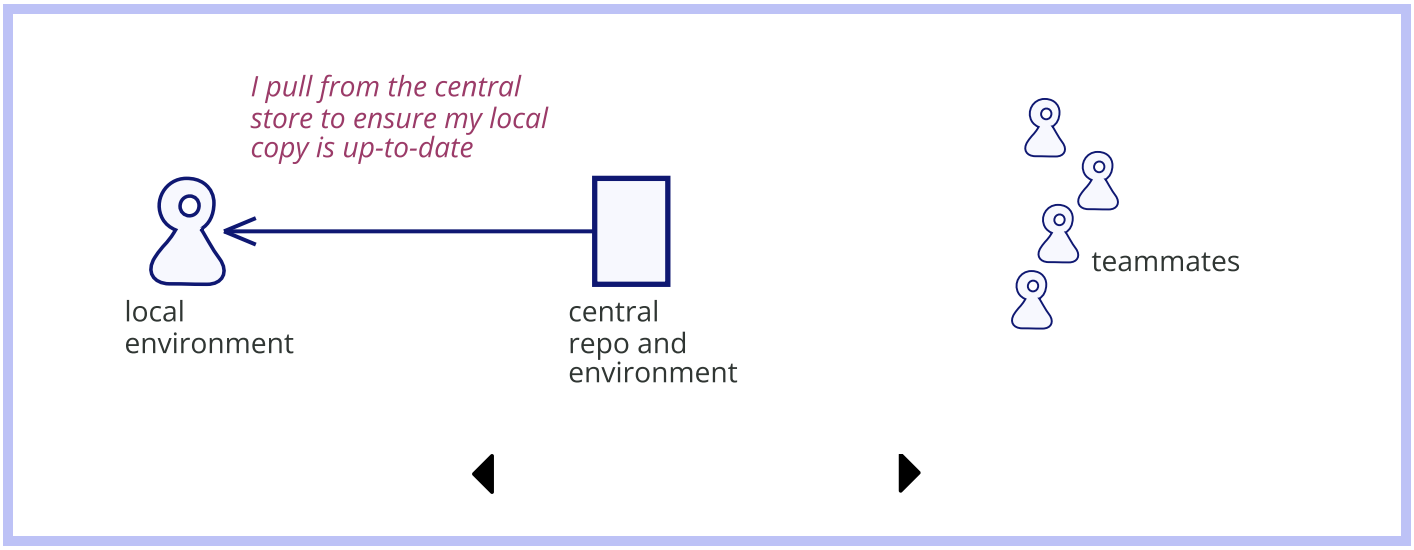
This contrast isn't the result of an expensive and complex tool. The essence of it lies in the simple practice of everyone on the team integrating frequently, at least daily, against a controlled source code repository. This practice is called “Continuous Integration” (or in some circles it’s called “Trunk-Based Development”).

In this article, I explain what Continuous Integration is and how to do it well. I've written it for two reasons. Firstly there are always new people coming into the profession and I want to show them how they can avoid that depressing warehouse. But secondly this topic needs clarity because Continuous Integration is a much misunderstood concept. There are many people who say that they are doing Continuous Integration, but once they describe their workflow, it becomes clear that they are missing important pieces. A clear understanding of Continuous Integration helps us communicate, so we know what to expect when we describe our way of working. It also helps folks realize that there are further things they can do to improve their experience.

I originally wrote this article in 2001, with an update in 2006. Since then much has changed in usual expectations of software development teams. The many-month integration that I saw in the 1980s is a distant memory, technologies such as version control and build scripts have become commonplace. I rewrote this article again in 2023 to better address the development teams of that time, with twenty years of experience to confirm the value of Continuous Integration.

Building a Feature with Continuous Integration

The easiest way for me to explain what Continuous Integration is and how it works is to show a quick example of how it works with the development of a small feature. I'm currently working with a major manufacturer of magic potions, we are extending their product quality system to calculate how long the potion's effect will last. We already have a dozen potions supported in the system, and we need to extend the logic for flying potions. (We've learned that having them wear off too early severely impacts customer retention.) Flying potions introduce a few new factors to take care of, one of which is the moon phase during secondary mixing.



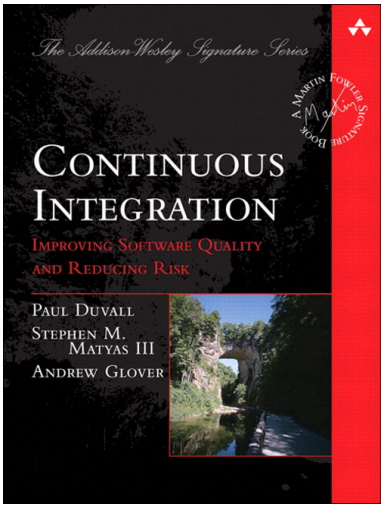
I begin by taking a copy of the latest product sources onto my local development environment. I do this by checking out the current mainline from the central repository with `git pull`.

Once the source is in my environment, I execute a command to build the product. This command checks that my environment is set up correctly, does any compilation of the sources into an executable product, starts the product, and runs a comprehensive suite of tests against it. This should take only a few minutes, while I start poking around the code to decide how to begin adding the new feature. This build hardly ever fails, but I do it just in case, because if it does fail, I want to know before I start making changes. If I make changes on top of a failing build, I'll get confused thinking it was my changes that caused the failure.

Now I take my working copy and do whatever I need to do to deal with the moon phases. This will consist of both altering the product code, and also adding or changing

some of the automated tests. During that time I run the automated build and tests frequently. After an hour or so I have the moon logic incorporated and tests updated.

Continuous Integration



A detailed handbook on the details of how to do Continuous Integration. After outlining the core principles of the practice, it explains how CI reduces risks by building software at every change. It then looks at a full-featured CI environment comprising continuous database integration, testing, inspection, deployment,and feedback.

by Paul Duvall, Steve Matyas, and Andrew Glover BOOK 2007

Read more...

I'm now ready to integrate my changes back into the central repository. My first step for this is to pull again, because it's possible, indeed likely, that my colleagues will have pushed changes into the mainline while I've been working. Indeed there are a couple of such changes, which I pull into my working copy. I combine my changes on top of them and run the build again. Usually this feels superfluous, but this time a test fails. The test gives me some clue about what's gone wrong, but I find it more useful to look at the commits that I pulled to see what changed. It seems that someone has made an adjustment to a function, moving some of its logic out into its callers. They fixed all the callers in the mainline code, but I added a new call in my changes that, of course, they

couldn't see yet. I make the same adjustment and rerun the build, which passes this time.

Since I was a few minutes sorting that out, I pull again, and again there's a new commit. However the build works fine with this one, so I'm able to `git push` my change up to the central repository.

However my push doesn't mean I'm done. Once I've pushed to the mainline a Continuous Integration Service notices my commit, checks out the changed code onto a CI agent, and builds it there. Since the build was fine in my environment I don't expect it to fail on the CI Service, but there is a reason that “works on my machine” is a well-known phrase in programmer circles. It's rare that something gets missed that causes the CI Services build to fail, but rare is not the same as never.

The integration machine's build doesn't take long, but it's long enough that an eager developer would be starting to think about the next step in calculating flight time. But I'm an old guy, so enjoy a few minutes to stretch my legs and read an email. I soon get a notification from the CI service that all is well, so I start the process again for the next part of the change.

Practices of Continuous Integration

The story above is an illustration of Continuous Integration that hopefully gives you a feel of what it's like for an ordinary programmer to work with. But, as with anything, there's quite a few things to sort out when doing this in daily work. So now we'll go through the key practices that we need to do.

Put everything in a version controlled mainline

These days almost every software team keeps their source code in a version control system, so that every developer can easily find not just the current state of the product, but all the changes that have been made to the product. Version control tools allow a system to be rolled back to any point in its development, which can be very

helpful to understand the history of the system, using Diff Debugging to find bugs. As I write this, the dominant version control system is git.

But while version control is commonplace, some teams fail to take full advantage of version control. My test for full version control is that I should be able to walk up with a very minimally configured environment - say a laptop with no more than the vanilla operating system installed - and be able to easily build, and run the product after cloning the repository. This means the repository should reliably return product source code, tests, database schema, test data, configuration files, IDE configurations, install scripts, third-party libraries, and any tools required to build the software.

> *I should be able to walk up with a laptop loaded with only an operating system, and by using the repository, obtain everything I need to build and run the product.*

You might notice I said that the repository should *return* all of these elements, which isn't the same as storing them. We don't have to store the compiler in the repository, but we need to be able to get at the right compiler. If I check out last year's product sources, I may need to be able to build them with the compiler I was using last year, not the version I'm using now. The repository can do this by storing a link to immutable asset storage - immutable in the sense that once an asset is stored with an id, I'll always get exactly that asset back again. I can also do this with library code, providing I both trust the asset storage and always reference a particular version, never “the latest version”.

Similar asset storage schemes can be used for anything too large, such as videos. Cloning a repository often means grabbing everything, even if it's not needed. By using references to an asset store, the build scripts can choose to download only what's needed for a particular build.

In general we should store in source control everything we need to build anything, but nothing that we actually build. Some people do keep the build products in source control, but I consider that to be a smell - an indication of a deeper problem, usually an inability to reliably recreate builds. It can be useful to cache build products, but they should always be treated as disposable, and it's usually good to then ensure they are removed promptly so that people don't rely on them when they shouldn't.

A second element of this principle is that it should be easy to find the code for a given piece of work. Part of this is clear names and URL schemes, both within the repository and within the broader enterprise. It also means not having to spend time figuring out which branch within the version control system to use. Continuous Integration relies on having a clear mainline - a single, shared, branch that acts as the current state of the product. This is the next version that will be deployed to production.

Teams that use git mostly use the name “main” for the mainline branch, but we also sometimes see “trunk” or the old default of “master”. The mainline is that branch on the central repository, so to add a commit to a mainline called `main` I need to first commit to my local copy of `main` and then push that commit to the central server. The tracking branch (called something like `origin/main`) is a copy of the mainline on my local machine. However it may be out of date, since in a Continuous Integration environment there are many commits pushed into mainline every day.

As much as possible, we should use text files to define the product and its environment. I say this because, although version-control systems can store and track non-text files, they don't usually provide any facility to easily see the difference between versions. This makes it much harder to understand what change was made. It's possible that in the future we'll see more storage formats having the facility to create meaningful diffs, but at the moment clear diffs are almost entirely reserved for text formats. Even there we need to use text formats that will produce comprehensible diffs.

Automate the Build

Turning the source code into a running system can often be a complicated process involving compilation, moving files around, loading schemas into databases, and so on. However like most tasks in this part of software development it can be automated - and as a result should be automated. Asking people to type in strange commands or clicking through dialog boxes is a waste of time and a breeding ground for mistakes.

> *Computers are designed to perform simple, repetitive tasks. As soon as you have humans doing repetitive tasks on behalf of computers, all the computers get together late at night and laugh at you.*
>
> -- *Neal Ford*

Most modern programming environments include tooling for automating builds, and such tools have been around for a long time. I first encountered them with make, one of the earliest Unix tools.

Any instructions for the build need to be stored in the repository, in practice this means that we must use text representations. That way we can easily inspect them to see how they work, and crucially, see diffs when they change. Thus teams using Continuous Integration avoid tools that require clicking around in UIs to perform a build or to configure an environment.

It's possible to use a regular programming language to automate builds, indeed simple builds are often captured as shell scripts. But as builds get more complicated it's better to use a tool that's designed with build automation in mind. Partly this is because such tools will have built-in functions for common build tasks. But the main reason is that build tools work best with a particular way to organize their logic - an alternative computational model that I refer to as a Dependency Network. A dependency network organizes its logic into tasks which are structured as a graph of dependencies.

A trivially simple dependency network might say that the “test” task is dependent upon the “compile” task. If I invoke the test task, it will look to see if the compile task needs to be run and if so invoke it first. Should the compile task itself have dependencies, the network will look to see if it needs to invoke them first, and so on backwards along the dependency chain. A dependency network like this is useful for build scripts because often tasks take a long time, which is wasted if they aren't needed. If nobody has changed any source files since I last ran the tests, then I can save doing a potentially long compilation.

To tell if a task needs to be run, the most common and straightforward way is to look at the modification times of files. If any of the input files to the compilation have been modified later than the output, then we know the compilation needs to be executed if that task is invoked.

A common mistake is not to include everything in the automated build. The build should include getting the database schema out of the repository and firing it up in the execution environment. I'll elaborate my earlier rule of thumb: anyone should be able to bring in a clean machine, check the sources out of the repository, issue a single command, and have a running system on their own environment.

While a simple program may only need a line or two of script file to build, complex systems often have a large graph of dependencies, finely tuned to minimize the amount

of time required to build things. This website, for example, has over a thousand web pages. My build system knows that should I alter the source for this page, I only have to build this one page. But should I alter a core file in the publication tool chain, then it needs to rebuild them all. Either way, I invoke the same command in my editor, and the build system figures out how much to do.

Depending on what we need, we may need different kinds of things to be built. We can build a system with or without test code, or with different sets of tests. Some components can be built stand-alone. A build script should allow us to build alternative targets for different cases.

Make the Build Self-Testing

Traditionally a build meant compiling, linking, and all the additional stuff required to get a program to execute. A program may run, but that doesn't mean it does the right thing. Modern statically typed languages can catch many bugs, but far more slip through that net. This is a critical issue if we want to integrate as frequently as Continuous Integration demands. If bugs make their way into the product, then we are faced with the daunting task of performing bug fixes on a rapidly-changing code base. Manual testing is too slow to cope with the frequency of change.

Faced with this, we need to ensure that bugs don't get into the product in the first place. The main technique to do this is a comprehensive test suite, one that is run before each integration to flush out as many bugs as possible. Testing isn't perfect, of course, but it can catch a lot of bugs - enough to be useful. Early computers I used did a visible memory self-test when they were booting up, which led me referring to this as Self Testing Code.

Writing self-testing code affects a programmer's workflow. Any programming task combines both modifying the functionality of the program, and also augmenting the test suite to verify this changed behavior. A programmer's job isn't done merely when the new feature is working, but also when they have automated tests to prove it.

Over the two decades since the first version of this article, I've seen programming environments increasingly embrace the need to provide the tools for programmers to build such test suites. The biggest push for this was JUnit, originally written by Kent Beck and Erich Gamma, which had a marked impact on the Java community in the late 1990s. This inspired similar testing frameworks for other languages, often referred to as Xunit frameworks. These stressed a light-weight, programmer-friendly mechanics

that allowed a programmer to easily build tests in concert with the product code. Often these tools have some kind of graphical progress bar that is green if the tests pass, but turns red should any fail - leading to phrases like “green build”, or “red-bar”.

> *A sound test suite would never allow a mischievous imp to do any damage without a test turning red.*

The test of such a test suite is that we should be confident that if the tests are green, then no significant bugs are in the product. I like to imagine a mischievous imp that is able to make simple modifications to the product code, such as commenting out lines, or reversing conditionals, but is not able to change the tests. A sound test suite would never allow the imp to do any damage without a test turning red. And any test failing is enough to fail the build, 99.9% green is still red.

Self-testing code is so important to Continuous Integration that it is a necessary prerequisite. Often the biggest barrier to implementing Continuous Integration is insufficient skill at testing.

That self-testing code and Continuous Integration are so tied together is no surprise. Continuous Integration was originally developed as part of Extreme Programming and testing has always been a core practice of Extreme Programming. This testing is often done in the form of Test Driven Development (TDD), a practice that instructs us to never write new code unless it fixes a test that we've written just before. TDD isn't essential for Continuous Integration, as tests can be written after production code as long as they are done before integration. But I do find that, most of the time, TDD is the best way to write self-testing code.

The tests act as an automated check of the health of the code base, and while tests are the key element of such an automated verification of the code, many programming environments provide additional verification tools. Linters can detect poor programming practices, and ensure code follows a team's preferred formatting style, vulnerability scanners can find security weaknesses. Teams should evaluate these tools to include them in the verification process.

Of course we can't count on tests to find everything. As it's often been said: tests don't prove the absence of bugs. However perfection isn't the only point at which we get

payback for a self-testing build. Imperfect tests, run frequently, are much better than perfect tests that are never written at all.

Everyone Pushes Commits To the Mainline Every Day

> *No code sits unintegrated for more than a couple of hours.*
>
> -- *Kent Beck*

Integration is primarily about communication. Integration allows developers to tell other developers about the changes they have made. Frequent communication allows people to know quickly as changes develop.

The one prerequisite for a developer committing to the mainline is that they can correctly build their code. This, of course, includes passing the build tests. As with any commit cycle the developer first updates their working copy to match the mainline, resolves any conflicts with the mainline, then builds on their local machine. If the build passes, then they are free to push to the mainline.

If everyone pushes to the mainline frequently, developers quickly find out if there's a conflict between two developers. The key to fixing problems quickly is finding them quickly. With developers committing every few hours a conflict can be detected within a few hours of it occurring, at that point not much has happened and it's easy to resolve. Conflicts that stay undetected for weeks can be very hard to resolve.

Semi-Integration (it isn't Continuous Integration)

There's a common confusion about the term “integration”. Many people who work on branches, perhaps a feature branch or a personal development branch, will regularly pull changes from mainline. They check that these don't break their work by building and testing after the pull. They may use a CI Service to do this, checking that any change to mainline doesn't break their branch.

But this isn't the full integration process. Full mainline integration requires that developers push their work back into the mainline. If they don't do that, then other team members can't see their work and check for any conflicts. This semi-integration doesn't prevent branches from diverging, conflicts from festering, and all the problems of low-frequency integration. Continuous Integration requires full mainline integration, no code sits on a branch longer than a few hours without being pushed into the mainline.

Conflicts in the codebase come in different forms. The easiest to find and resolve are textual conflicts, often called “merge conflicts”, when two developers edit the same fragment of code in different ways. Version-control tools detect these easily once the second developer pulls the updated mainline into their working copy. The harder problem are Semantic Conflicts. If my colleague changes the name of a function and I call that function in my newly added code, the version-control system can't help us. In a statically typed language we get a compilation failure, which is pretty easy to detect, but in a dynamic language we get no such help. And even statically-typed compilation doesn't help us when a colleague makes a change to the body of a function that I call, making a subtle change to what it does. This is why it's so important to have self-testing code.

A test failure alerts that there's a conflict between changes, but we still have to figure out what the conflict is and how to resolve it. Since there's only a few hours of changes between commits, there's only so many places where the problem could be hiding. Furthermore since not much has changed we can use Diff Debugging to help us find the bug.

My general rule of thumb is that every developer should commit to the mainline every day. In practice, those experienced with Continuous Integration integrate more frequently than that. The more frequently we integrate, the less places we have to look for conflict errors, and the more rapidly we fix conflicts.

Frequent commits encourage developers to break down their work into small chunks of a few hours each. This helps track progress and provides a sense of progress. Often people initially feel they can't do something meaningful in just a few hours, but we've found that mentoring and practice helps us learn.

Every Push to Mainline Should Trigger a Build

If everyone on the team integrates at least daily, this ought to mean that the mainline stays in a healthy state. In practice, however, things still do go wrong. This may be due to lapses in discipline, neglecting to update and build before a push, there may also be environmental differences between developer workspaces.

We thus need to ensure that every commit is verified in a reference environment. The usual way to do this is with a **Continuous Integration Service (CI Service)** that monitors the mainline. (Examples of CI Services are tools like Jenkins, GitHub Actions, Circle CI etc.) Every time the mainline receives a commit, the CI service checks out the

head of the mainline into an integration environment and performs a full build. Only once this integration build is green can the developer consider the integration to be complete. By ensuring we have a build with every push, should we get a failure, we know that the fault lies in that latest push, narrowing down where have to look to fix it.

I want to stress here that when we use a CI Service, we only use it on the mainline, which is the main branch on the reference instance of the version control system. It's common to use a CI service to monitor and build from multiple branches, but the whole point of integration is to have all commits coexisting on a *single* branch. While it may be useful to use CI service to do an automated build for different branches, that's not the same as Continuous Integration, and teams using Continuous Integration will only need the CI service to monitor a single branch of the product.

While almost all teams use CI Services these days, it is perfectly possible to do Continuous Integration without one. Team members can manually check out the head on the mainline onto an integration machine and perform a build to verify the integration. But there's little point in a manual process when automation is so freely available.

(This is an appropriate point to mention that my colleagues at Thoughtworks, have contributed a lot of open-source tooling for Continuous Integration, in particular Cruise Control - the first CI Service.)

Fix Broken Builds Immediately

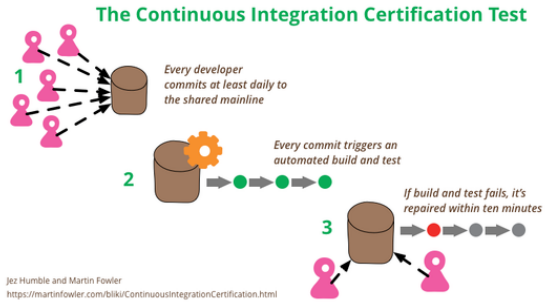
Continuous Integration can only work if the mainline is kept in a healthy state. Should the integration build fail, then it needs to be fixed right away. As Kent Beck puts it: “nobody has a higher priority task than fixing the build”. This doesn't mean that everyone on the team has to stop what they are doing in order to fix the build, usually it only needs a couple of people to get things working again. It does mean a conscious prioritization of a build fix as an urgent, high priority task

> *Usually the best way to fix the build is to revert the faulty commit from the mainline, allowing the rest of the team to continue working.*

Usually the best way to fix the build is to revert the latest commit from the mainline, taking the system back to the last-known good build. If the cause of the problem is immediately obvious then it can be fixed directly with a new commit, but otherwise reverting the mainline allows some folks to figure out the problem in a separate development environment, allowing the rest of the team to continue to work with the mainline.

Some teams prefer to remove all risk of breaking the mainline by using a Pending Head (also called Pre-tested, Delayed, or Gated Commit.) To do this the CI service needs to set things up so that commits pushed to the mainline for integration do not immediately go onto the mainline. Instead they are placed on another branch until the build completes and only migrated to the mainline after a green build. While this technique avoids any danger to mainline breaking, an effective team should rarely see a red mainline, and on the few times it happens its very visibility encourages folks to learn how to avoid it.

Continuous Integration Certification



Jez Humble's simple three-part test for Continuous Integration. Jez regularly uses it during conference talks, demonstrating that most people that claim to be doing Continuous Integration actually are not.

by Martin Fowler BLIKI 18 Jan 2017

Read more...

CERTIFICATION CONTINUOUS DELIVERY

Keep the Build Fast

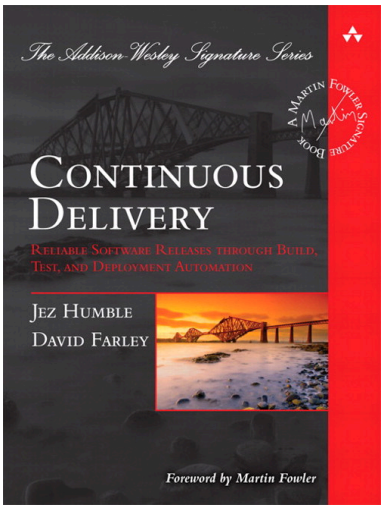
The whole point of Continuous Integration is to provide rapid feedback. Nothing sucks the blood of Continuous Integration more than a build that takes a long time. Here I must admit a certain crotchety old guy amusement at what's considered to be a long build. Most of my colleagues consider a build that takes an hour to be totally unreasonable. I remember teams dreaming that they could get it so fast - and occasionally we still run into cases where it's very hard to get builds to that speed.

For most projects, however, the XP guideline of a ten minute build is perfectly within reason. Most of our modern projects achieve this. It's worth putting in concentrated effort to make it happen, because every minute chiseled off the build time is a minute saved for each developer every time they commit. Since Continuous Integration demands frequent commits, this adds up to a lot of the time.

If we're staring at a one hour build time, then getting to a faster build may seem like a daunting prospect. It can even be daunting to work on a new project and think about how to keep things fast. For enterprise applications, at least, we've found the usual bottleneck is testing - particularly tests that involve external services such as a database.

Probably the most crucial step is to start working on setting up a Deployment Pipeline. The idea behind a **deployment pipeline** (also known as **build pipeline** or **staged build**) is that there are in fact multiple builds done in sequence. The commit to the mainline triggers the first build - what I call the commit build. The **commit build** is the build that's needed when someone pushes commits to the mainline. The commit build is the one that has to be done quickly, as a result it will take a number of shortcuts that will reduce the ability to detect bugs. The trick is to balance the needs of bug finding and speed so that a good commit build is stable enough for other people to work on.

Continuous Delivery



The definitive book on Continuous Delivery, which outlines the practices needed to bring code into production rapidly and safely. The key aspects are collaboration between everyone involved in the release process and automating as many aspects of that process as you can. The book goes through the foundations of configuration management, automated testing, and continuous integration - on which it shows how to build deployment pipelines to take integrated, tested code live. The book details the delivery ecosystem, managing infrastructure, environments and data.

by Jez Humble and David Farley	BOOK 2010

Read more...

Once the commit build is good then other people can work on the code with confidence. However there are further, slower, tests that we can start to do. Additional machines can run further testing routines on the build that take longer to do.

A simple example of this is a two stage deployment pipeline. The first stage would do the compilation and run tests that are more localized unit tests with slow services replaced by Test Doubles, such as a fake in-memory database or a stub for an external service. Such tests can run very fast, keeping within the ten minute guideline. However any bugs that involve larger scale interactions, particularly those involving the real database, won't be found. The second stage build runs a different suite of tests that do hit a real database and involve more end-to-end behavior. This suite might take a couple of hours to run.

In this scenario people use the first stage as the commit build and use this as their main CI cycle. If the secondary build fails, then this may not have the same 'stop everything' quality, but the team does aim to fix such bugs as rapidly as possible, while keeping the commit build running. Since the secondary build may be much slower, it may not run after every commit. In that case it runs as often as it can, picking the last good build from the commit stage.

If the secondary build detects a bug, that's a sign that the commit build could do with another test. As much as possible we want to ensure that any later-stage failure leads to new tests in the commit build that would have caught the bug, so the bug stays fixed in the commit build. This way the commit tests are strengthened whenever something gets past them. There are cases where there's no way to build a fast-running test that exposes the bug, so we may decide to only test for that condition in the secondary build. Most of the time, fortunately, we can add suitable tests to the commit build.

Another way to speed things up is to use parallelism and multiple machines. Cloud environments, in particular, allow teams to easily spin up a small fleet of servers for builds. Providing the tests can run reasonably independently, which well-written tests can, then using such a fleet can get very rapid build times. Such parallel cloud builds may also be worthwhile to a developer's pre-integration build too.

While we're considering the broader build process, it's worth mentioning another category of automation, interaction with dependencies. Most software uses a large range of dependent software produced by different organizations. Changes in these dependencies can cause breakages in the product. A team should thus automatically check for new versions of dependencies and integrate them into the build, essentially as if they were another team member. This should be done frequently, usually at least daily, depending on the rate of change of the dependencies. A similar approach should be used with running Contract Tests. If these dependency interactions go red, they don't have the same “stop the line” effect as regular build failures, but do require prompt action by the team to investigate and fix.

Hide Work-in-Progress

Continuous Integration means integrating as soon as there is a little forward progress and the build is healthy. Frequently this suggests integrating before a user-visible feature is fully formed and ready for release. We thus need to consider how to deal with latent code: code that's part of an unfinished feature that's present in a live release.

Some people worry about latent code, because it's putting non-production quality code into the released executable. Teams doing Continuous Integration ensure that all code sent to the mainline is production quality, together with the tests that verify the code. Latent code may never be executed in production, but that doesn't stop it from being exercised in tests.

We can prevent the code being executed in production by using a Keystone Interface - ensuring the interface that provides a path to the new feature is the last thing we add to the code base. Tests can still check the code at all levels other than that final interface. In a well-designed system, such interface elements should be minimal and thus simple to add with a short programming episode.

Using Dark Launching we can test some changes in production before we make them visible to the user. This technique is useful for assessing the impact on performance,

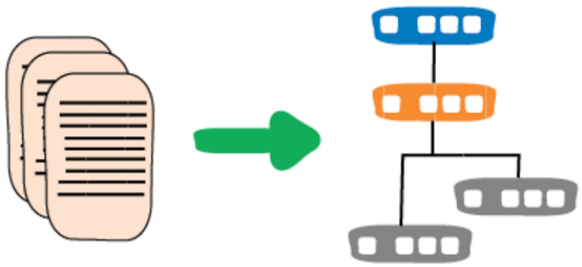
Keystones cover most cases of latent code, but for occasions where that's not possible we use Feature Flags. Feature flags are checked whenever we are about to execute latent code, they are set as part of the environment, perhaps in an environment-specific configuration file. That way the latent code can be active for testing, but disabled in production. As well as enabling Continuous Integration, feature flags also make it easier for runtime switching for A/B testing and Canary Releases. We then make sure we remove this logic promptly once a feature is fully released, so that the flags don't clutter the code base.

Branch By Abstraction is another technique for managing latent code, which is particularly useful for large infrastructural changes within a code base. Essentially this creates an internal interface to the modules that are being changed. The interface can then route between old and new logic, gradually replacing execution paths over time. We've seen this done to switch such pervasive elements as changing the persistence platform.

When introducing a new feature, we should always ensure that we can rollback in case of problems. Parallel Change (aka expand-contract) breaks a change into reversible steps. For example, if we rename a database field, we first create a new field with the new name, then write to both old and new fields, then copy data from the exisitng old fields, then read from the new field, and only then remove the old field. We can reverse any of these steps, which would not be possible if we made such a change all at once. Teams using Continuous Integration often look to break up changes in this way, keeping changes small and easy to undo.

Test in a Clone of the Production Environment

Infrastructure As Code



Infrastructure as code is the approach to defining computing and network infrastructure through source code that can then be treated just like any software system. Such code can be kept in source control to allow auditability and ReproducibleBuilds, subject to testing practices, and the full discipline of ContinuousDelivery. It's an approach that's been used over the last decade to deal with growing CloudComputing platforms and will become the dominant way to handle computing infrastructure in the next.

by Martin Fowler BLIKI 1 Mar 2016

Read more...

CONTINUOUS DELIVERY MICROSERVICES

The point of testing is to flush out, under controlled conditions, any problem that the system will have in production. A significant part of this is the environment within which the production system will run. If we test in a different environment, every difference results in a risk that what happens under test won't happen in production.

Consequently, we want to set up our test environment to be as exact a mimic of our production environment as possible. Use the same database software, with the same versions, use the same version of the operating system. Put all the appropriate libraries that are in the production environment into the test environment, even if the system doesn't actually use them. Use the same IP addresses and ports, run it on the same hardware.

Virtual environments make it much easier than it was in the past to do this. We run production software in containers, and reliably build exactly the same containers for testing, even in a developer's workspace. It's worth the effort and cost to do this, the price is usually small compared to hunting down a single bug that crawled out of the hole created by environment mismatches.

Some software is designed to run in multiple environments, such as different operating systems and platform versions. The deployment pipeline should arrange for testing in all of these environments in parallel.

A point to take care of is when the production environment isn't as good as the development environment. Will the production software be running on machines connected with dodgy wifi, like smartphones? Then ensure a test environment mimics poor network connections.

Everyone can see what's happening

Continuous Integration is all about communication, so we want to ensure that everyone can easily see the state of the system and the changes that have been made to it.

One of the most important things to communicate is the state of the mainline build. CI Services have dashboards that allow everyone to see the state of any builds they are running. Often they link with other tools to broadcast build information to internal social media tools such as Slack. IDEs often have hooks into these mechanisms, so developers can be alerted while still inside the tool they are using for much of their work. Many teams only send out notifications for build failures, but I think it's worth sending out messages on success too. That way people get used to the regular signals and get a sense for the length of the build. Not to mention the fact that it's nice to get a “well done” every day, even if it's only from a CI server.

Teams that share a physical space often have some kind of always-on physical display for the build. Usually this takes the form of a large screen showing a simplified dashboard. This is particularly valuable to alert everyone to a broken build, often using the red/green colors on the mainline commit build.

> One of the older physical displays I rather liked were the use of red and green lava lamps. One of the features of a lava lamp is that after they are turned on for a while they start to bubble. The idea was that if the red lamp came on, the team should fix the build before it starts to bubble. Physical displays for

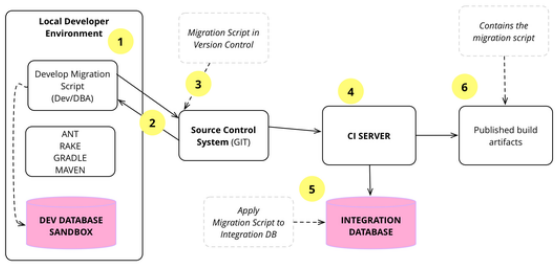
build status often got playful, adding some quirky personality to a team's workspace. I have fond memories of a dancing rabbit.

As well as the current state of the build, these displays can show useful information about recent history, which can be an indicator of project health. Back at the turn of the century I worked with a team who had a history of being unable to create stable builds. We put a calendar on the wall that showed a full year with a small square for each day. Every day the QA group would put a green sticker on the day if they had received one stable build that passed the commit tests, otherwise a red square. Over time the calendar revealed the state of the build process showing a steady improvement until green squares were so common that the calendar disappeared - its purpose fulfilled.

Automate Deployment

To do Continuous Integration we need multiple environments, one to run commit tests, probably more to run further parts of the deployment pipeline. Since we are moving executables between these environments multiple times a day, we'll want to do this automatically. So it's important to have scripts that will allow us to deploy the application into any environment easily.

Evolutionary Database Design



Over the last decade we've developed and refined a number of techniques that allow a database design to evolve as an application develops. This is a very important capability for agile methodologies. The techniques rely on applying continuous integration and automated refactoring to database development, together with a close collaboration between DBAs and application developers. The techniques work in

both pre-production and released systems, in green field projects as well as legacy systems.

by Pramod Sadalage and Martin Fowler ARTICLE May 2016

Read more...

AGILE REFACTORING APPLICATION ARCHITECTURE DATABASE EVOLUTIONARY DESIGN

With modern tools for virtualization, containerization, and serverless we can go further. Not just have scripts to deploy the product, but also scripts to build the required environment from scratch. This way we can start with a bare-bones environment that's available off-the-shelf, create the environment we need for the product to run, install the product, and run it - all entirely automatically. If we're using feature flags to hide work-in-progress, then these environments can be set up with all the feature-flags on, so these features can be tested with all immanent interactions.

A natural consequence of this is that these same scripts allow us to deploy into production with similar ease. Many teams deploy new code into production multiple times a day using these automations, but even if we choose a less frequent cadence, automatic deployment helps speed up the process and reduces errors. It's also a cheap option since it just uses the same capabilities that we use to deploy into test environments.

If we deploy into production automatically, one extra capability we find handy is automated rollback. Bad things do happen from time to time, and if smelly brown substances hit rotating metal, it's good to be able to quickly go back to the last known good state. Being able to automatically revert also reduces a lot of the tension of deployment, encouraging people to deploy more frequently and thus get new features out to users quickly. Blue Green Deployment allows us to both make new versions live quickly, and to roll back equally quickly if needed, by shifting traffic between deployed versions.

Automated Deployment make it easier to set up Canary Releases, deploying a new version of a product to a subset of our users in order to flush out problems before releasing to the full population.

Mobile applications are good examples of where it's essential to automate deployment into test environments, in this case onto devices so that a new version can be explored

before invoking the guardians of the App Store. Indeed any device-bound software needs ways to easily get new versions on to test devices.

When deploying software like this, remember to ensure that version information is visible. An about screen should contain a build id that ties back to version control, logs should make it easy to see which version of the software is running, there should be some API endpoint that will give version information.

Styles of Integration

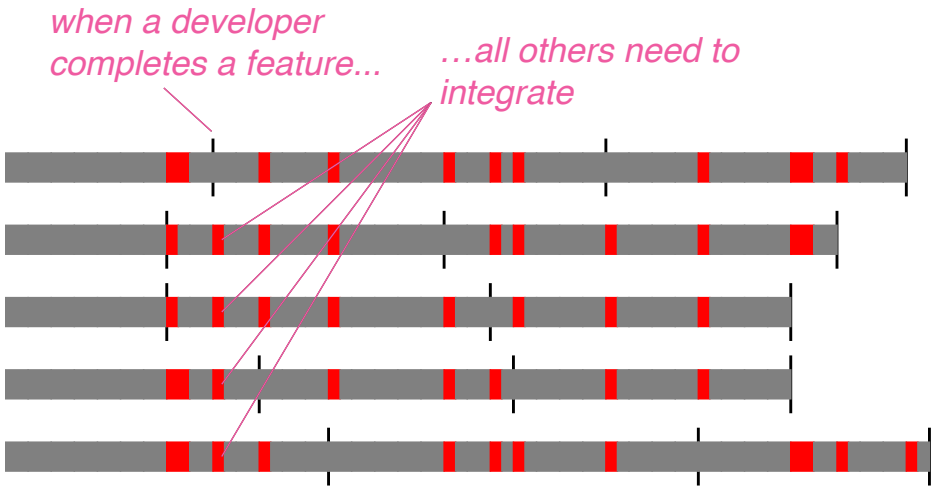
Thus far, I've described one way to approach integration, but if it's not universal, then there must be other ways. As with anything, any classification I give has fuzzy boundaries, but I find it useful to think of three styles of handling integration: Pre-Release Integration, Feature Branches, and Continuous Integration.

The oldest is the one I saw in that warehouse in the 80's - **Pre-Release Integration**. This sees integration as a phase of a software project, a notion that is a natural part of a Waterfall Process. In such a project work is divided into units, which may be done by individuals or small teams. Each unit is a portion of the software, with minimal interaction with other units. These units are built and tested on their own (the original use of the term “unit test”). Then once the units are ready, we integrate them into the final product. This integration occurs once, and is followed by integration testing, and on to a release. Thus if we think of the work, we see two phases, one where everyone works in parallel on features, followed by a single stream of effort at integration.

The frequency of integration in this style is tied to the frequency of release, usually major versions of the software, usually measured in months or years. These teams will use a different process for urgent bug fixes, so they can be released separately to the regular integration schedule.

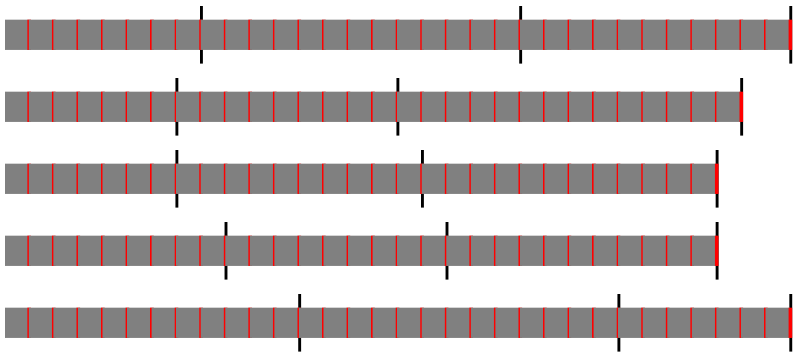
One of the most popular approaches to integration these days is to use **Feature Branches**. In this style features are assigned to individuals or small teams, much as units in the older approach. However, instead of waiting until all the units are done before integrating, developers integrate their feature into the mainline as soon as it's done. Some teams will release to production after each feature integration, others prefer to batch up a few features for release.

Teams using feature branches will usually expect everyone to pull from mainline regularly, but this is semi-integration. If Rebecca and I are working on separate features, we might pull from mainline every day, but we don't see each other's changes until one of us completes our feature and integrates, pushing it to the mainline. Then the other will see that code on their next pull, integrating it into their working copy. Thus after each feature is pushed to mainline, every other developer will then do integration work to combine this latest mainline push with their own feature branch.



This is only semi-integration because each developer combines the changes on mainline to their own local branch. Full integration can't happen until a developer pushes their changes, causing another round of semi-integrations. Even if Rebecca and I both pull the same changes from mainline, we've only integrated with those changes, not with each other's branches.

With **Continuous Integration**, every day we are all pushing our changes to the mainline and pulling everyone else's changes into our own work. This leads to many more bouts of integration work, but each bout is much smaller. It's much easier to combine a few hours work on a code base than to combine several days.



Benefits of Continuous Integration

When discussing the relative merits of the three styles of integration, most of the discussion is truly about the frequency of integration. Both Pre-Release Integration and Feature Branching can operate at different frequencies and it's possible to change integration frequency without changing the style of integration. If we're using Pre-Release Integration, there's a big difference between monthly releases and annual releases. Feature Branching usually works at a higher frequency, because integration occurs when each feature is individually pushed to mainline, as opposed to waiting to batch a bunch of units together. If a team is doing Feature Branching and all its features are less than a day's work to build, then they are effectively the same as Continuous Integration. But Continuous Integration is different in that it's *defined* as a high-frequency style. Continuous Integration makes a point of setting integration frequency as a target in itself, and not binding it to feature completion or release frequency.

It thus follows that most teams can see a useful improvement in the factors I'll discuss below by increasing their frequency without changing their style. There are significant benefits to reducing the size of features from two months to two weeks. Continuous Integration has the advantage of setting high-frequency integration as the baseline, setting habits and practices that make it sustainable.

Reduced risk of delivery delays

It's very hard to estimate how long it takes to do a complex integration. Sometimes it can be a struggle to merge in git, but then all works well. Other times it can be a quick merge, but a subtle integration bug takes days to find and fix. The longer the time between integrations, the more code to integrate, the longer it takes - but what's worse is the increase in unpredictability.

This all makes pre-release integration a special form of nightmare. Because the integration is one of the last steps before release, time is already tight and the pressure is on. Having a hard-to-predict phase late in the day means we have a significant risk that's very difficult to mitigate. That was why my 80's memory is so strong, and it's hardly the only time I've seen projects stuck in an integration hell, where every time they fix an integration bug, two more pop up.

Any steps to increase integration frequency lowers this risk. The less integration there is to do, the less unknown time there is before a new release is ready. Feature Branching helps by pushing this integration work to individual feature streams, so that, if left alone, a stream can push to mainline as soon as the feature is ready.

But that *left alone* point is important. If anyone else pushes to mainline, then we introduce some integration work before the feature is done. Because the branches are isolated, a developer working on one branch doesn't have much visibility about what other features may push, and how much work would be involved to integrate them. While there is a danger that high priority features can face integration delays, we can manage this by preventing pushes of lower-priority features.

Continuous Integration effectively eliminates delivery risk. The integrations are so small that they usually proceed without comment. An awkward integration would be one that takes more than a few minutes to resolve. The very worst case would be conflict that causes someone to restart their work from scratch, but that would still be less than a day's work to lose, and is thus not going to be something that's likely to trouble a board of stakeholders. Furthermore we're doing integration regularly as we develop the software, so we can face problems while we have more time to deal with them and can practice how to resolve them.

Even if a team isn't releasing to production regularly, Continuous Integration is important because it allows everyone to see exactly what the state of the product is. There's no hidden integration efforts that need to be done before release, any effort in integration is already baked in.

Less time wasted in integration

I've not seen any serious studies that measure how time spent on integration matches the size of integrations, but my anecdotal evidence strongly suggests that the relationship isn't linear. If there's twice as much code to integrate, it's more likely to be four times as long to carry out the integration. It's rather like how we need three lines to fully connect three nodes, but six lines to connect four of them. Integration is all about connections, hence the non-linear increase, one that's reflected in the experience of my colleagues.

In organizations that are using feature branches, much of this lost time is felt by the individual. Several hours spent trying to rebase on a big change to mainline is frustrating. A few days spent waiting for a code review on a finished pull request, which another big mainline change during the waiting period is even more frustrating. Having to put work on a new feature aside to debug a problem found in an integration test of feature finished two weeks ago saps productivity.

When we're doing Continuous Integration, integration is generally a non-event. I pull down the mainline, run the build, and push. If there is a conflict, the small amount of code I've written is fresh in my mind, so it's usually easy to see. The workflow is regular, so we're practiced at it, and we're incentives to automate it as much as possible.

Like many of these non-linear effects, integration can easily become a trap where people learn the wrong lesson. A difficult integration may be so traumatic that a team decides it should do integrations less often, which only exacerbates the problem in the future.

What's happening here is that we are seeing much closer collaboration between the members of the team. Should two developers make decisions that conflict, we find out when we integrate. So the less time between integrations, the less time before we detect the conflict, and we can deal with the conflict before it grows too big. With high-frequency integration, our source control system becomes a communication channel, one that can communicate things that can otherwise be unsaid.

Less Bugs

Bugs - these are the nasty things that destroy confidence and mess up schedules and reputations. Bugs in deployed software make users angry with us. Bugs cropping up during regular development get in our way, making it harder to get the rest of the software working correctly.

Continuous Integration doesn't get rid of bugs, but it does make them dramatically easier to find and remove. This is less because of the high-frequency integration and more due to the essential introduction of self-testing code. Continuous Integration doesn't work without self-testing code because without decent tests, we can't keep a healthy mainline. Continuous Integration thus institutes a regular regimen of testing. If the tests are inadequate, the team will quickly notice, and can take corrective action. If a bug appears due to a semantic conflict, it's easy to detect because there's only a small amount of code to be integrated. Frequent integrations also work well with Diff Debugging, so even a bug noticed weeks later can be narrowed down to a small change.

Bugs are also cumulative. The more bugs we have, the harder it is to remove each one. This is partly because we get bug interactions, where failures show as the result of multiple faults - making each fault harder to find. It's also psychological - people have less energy to find and get rid of bugs when there are many of them. Thus self-testing

code reinforced by Continuous Integration has another exponential effect in reducing the problems caused by defects.

This runs into another phenomenon that many people find counter-intuitive. Seeing how often introducing a change means introducing bugs, people conclude that to have high reliability software they need to slow down the release rate. This was firmly contradicted by the DORA research program led by Nicole Forsgren. They found that elite teams deployed to production more rapidly, more frequently, and had a dramatically lower incidence of failure when they made these changes. The research also finds that teams have higher levels of performance when they have three or fewer active branches in the application's code repository, merge branches to mainline at least once a day, and don't have code freezes or integration phases.

Enables Refactoring for sustained productivity

Most teams observe that over time, codebases deteriorate. Early decisions were good at the time, but are no longer optimal after six month's work. But changing the code to incorporate what the team has learned means introducing changes deep in the existing code, which results in difficult merges which are both time-consuming and full of risk. Everyone recalls that time someone made what would be a good change for the future, but caused days of effort breaking other people's work. Given that experience, nobody wants to rework the structure of existing code, even though it's now awkward for everyone to build on, thus slowing down delivery of new features.

Refactoring is an essential technique to attenuate and indeed reverse this process of decay. A team that refactors regularly has a disciplined technique to improve the structure of a code base by using small, behavior-preserving transformations of the code. These characteristics of the transformations greatly reduce their chances of introducing bugs, and they can be done quickly, especially when supported by a foundation of self-testing code. Applying refactoring at every opportunity, a team can improve the structure of an existing codebase, making it easier and faster to add new capabilities.

But this happy story can be torpedoed by integration woes. A two week refactoring session may greatly improve the code, but result in long merges because everyone else has been spending the last two weeks working with the old structure. This raises the costs of refactoring to prohibitive levels. Frequent integration solves this dilemma by ensuring that both those doing the refactoring and everyone else are regularly synchronizing their work. When using Continuous Integration, if someone makes

intrusive changes to a core library I'm using, I only have to adjust a few hours of programming to these changes. If they do something that clashes with the direction of my changes, I know right away, so have the opportunity to talk to them so we can figure out a better way forward.

So far in this article I've raised several counter-intuitive notions about the merits of high-frequency integration: that the more often we integrate, the less time we spend integrating, and that frequent integration leads to less bugs. Here is perhaps the most important counter-intuitive notion in software development: that teams that spend a lot of effort keeping their code base healthy deliver features faster and cheaper. Time invested in writing tests and refactoring delivers impressive returns in delivery speed, and Continuous Integration is a core part of making that work in a team setting.

Release to Production is a business decision

Imagine we are demonstrating some newly built feature to a stakeholder, and she reacts by saying - “this is really cool, and would make a big business impact. How long before we can make this live?” If that feature is being shown on an unintegrated branch, then the answer may be weeks or months, particularly if there is poor automation on the path to production. Continuous Integration allows us to maintain a Release-Ready Mainline, which means the decision to release the latest version of the product into production is purely a business decision. If the stakeholders want the latest to go live, it's a matter of minutes running an automated pipeline to make it so. This allows the customers of the software greater control of when features are released, and encourages them to collaborate more closely with the development team

Continuous Integration and a Release-Ready Mainline removes one of the biggest barriers to frequent deployment. Frequent deployment is valuable because it allows our users to get new features more rapidly, to give more rapid feedback on those features, and generally become more collaborative in the development cycle. This helps break down the barriers between customers and development - barriers which I believe are the biggest barriers to successful software development.

When we should *not* use Continuous Integration

All those benefits sound rather juicy. But folks as experienced (or cynical) as I am are always suspicious of a bare list of benefits. Few things come without a cost, and decisions about architecture and process are usually a matter of trade-offs.

But I confess that Continuous Integration is one of those rare cases where there's little downside for a committed and skillful team to utilize it. The cost imposed by sporadic integration is so great, that almost any team can benefit by increasing their integration frequency. There is some limit to when the benefits stop piling up, but that limit sits at hours rather than days, which is exactly the territory of Continuous Integration. The interplay between self-testing code, Continuous Integration, and Refactoring is particularly strong. We've been using this approach for two decades at Thoughtworks, and our only question is how to do it more effectively - the core approach is proven.

But that doesn't mean that Continuous Integration is for everyone. You might notice that I said that “there’s little downside for a *committed and skillful* team to utilize it”. Those two adjectives indicate the contexts where Continuous Integration isn't a good fit.

By “committed”, I mean a team that's working full-time on a product. A good counter-example to this is a classical open-source project, where there is one or two maintainers and many contributors. In such a situation even the maintainers are only doing a few hours a week on the project, they don't know the contributors very well, and don't have good visibility for when contributors contribute or the standards they should follow when they do. This is the environment that led to a feature branch workflow and pull-requests. In such a context Continuous Integration isn't plausible, although efforts to increase the integration frequency can still be valuable.

Continuous Integration is more suited for team working full-time on a product, as is usually the case with commercial software. But there is much middle ground between the classical open-source and the full-time model. We need to use our judgment about what integration policy to use that fits the commitment of the team.

The second adjective looks at the skill of the team in following the necessary practices. If a team attempts Continuous Integration without a strong test suite, they will run into all sorts of trouble because they don't have a mechanism for screening out bugs. If they don't automate, integration will take too long, interfering with the flow of development. If folks aren't disciplined about ensuring their pushes to mainline are

done with green builds, then the mainline will end up broken all the time, getting in the way of everyone's work.

Anyone who is considering introducing Continuous Integration has to bear these skills in mind. Instituting Continuous Integration without self-testing code won't work, and it will also give a inaccurate impression of what Continuous Integration is like when it's done well.

That said, I don't think the skill demands are particularly hard. We don't need rock-star developers to get this process working in a team. (Indeed rock-star developers are often a barrier, as people who think of themselves that way usually aren't very disciplined.) The skills for these technical practices aren't that hard to learn, usually the problem is finding a good teacher, and forming the habits that crystallize the discipline. Once the team gets the hang of the flow, it usually feels comfortable, smooth - and fast.

Introducing Continuous Integration

One of the hard things about describing how to introduce a practice like Continuous Integration is that the path depends very much on where you're starting. Writing this, I don't know what kind code you are working on, what skills and habits your team possesses, let alone the broader organizational context. All anyone like me can do is point out some common signposts, in the hope that it will help you find your own path.

When introducing any new practice, it's important to be clear on why we're doing it. My list of benefits above includes the most common reasons, but different contexts lead to a different level of importance for them. Some benefits are harder to appreciate than others. Reducing waste in integration addresses a frustrating problem, and can be easily sensed as we make progress. Enabling refactoring to reduce the cruft in a system and improve overall productivity is more tricky to see. It takes time before we see an effect, and it's hard to compare to the counter-factual. Yet this is probably the most valuable benefit of Continuous Integration.

The list of practices above indicate the skills a team needs to learn in order to make Continuous Integration work. Some of these can bring value even before we get close

to the high integration frequency. Self-testing code adds stability to a system even with infrequent commits.

One target can be to double the integration frequency. If feature branches typically run for ten days, figure out how to cut them down to five. This may involve better build and test automation, and creative thinking on how a large task can be split into smaller, independently integrated tasks. If we use pre-integration reviews, we could include explicit steps in those reviews to check test coverage and encourage smaller commits.

If you are starting a new project, we can begin with Continuous Integration from the beginning. We should keep an eye on build times and take action as soon as we start going slower than the ten minute rule. By acting quickly we'll make the necessary restructurings before the code base gets so big that it becomes a major pain.

Above all we should get some help. We should find someone who has done Continuous Integration before to help us. Like any new technique it's hard to introduce it when we don't know what the final result looks like. It may cost money to get this support, but we'll otherwise pay in lost time and productivity. (Disclaimer / Advert - yes we at Thoughtworks do some consultancy in this area. After all we've made most of the mistakes that there are to make.)

Common Questions

Where did Continuous Integration come from?

Continuous Integration was developed as a practice by Kent Beck as part of Extreme Programming in the 1990s. At that time pre-release integration was the norm, with release frequencies often measured in years. There had been a general push to iterative development, with faster release cycles. But few teams were thinking in weeks between releases. Kent defined the practice, developed it with projects he worked on, and established how it interacted with the other key practices upon which it relies.

> Microsoft had been known for doing daily builds (usually overnight), but without the testing regimen or the focus on fixing defects that are such crucial elements of Continuous Integration.

Some people credit Grady Booch for coining the term, but he only used the phrase as an offhand description in a single sentence in his object-oriented design book. He did not treat it as a defined practice, indeed it didn't appear in the index.

What is the difference between Continuous Integration and Trunk-Based Development?

As CI Services became popular, many people used them to run regular builds on feature branches. This, as explained above, isn't Continuous Integration at all, but it led to many people saying (and thinking) they were doing Continuous Integration when they were doing something significantly different, which causes a lot of confusion.

Some folks decided to tackle this Semantic Diffusion by coining a new term: Trunk-Based Development. In general I see this as a synonym to Continuous Integration and acknowledge that it doesn't tend to suffer from confusion with "running Jenkins on our feature branches". I've read some people trying to formulate some distinction between the two, but I find these distinctions are neither consistent nor compelling.

I don't use the term Trunk-Based Development, partly because I don't think coining a new name is a good way to counter semantic diffusion, but mostly because renaming this technique rudely erases the work of those, especially Kent Beck, who championed and developed Continuous Integration in the beginning.

Despite me avoiding the term, there is a lot of good information about Continuous Integration that's written under the flag of Trunk-Based Development. In particular, Paul Hammant has written a lot of excellent material on his website.

Can we run a CI Service on our feature branches?

The simple answer is "yes - but you're *not* doing Continuous Integration". The key principle here is that "Everyone Commits To the Mainline Every Day". Doing an automated build on feature branches is useful, but it is only semi-integration.

However it is a common confusion that using a daemon build in this way is what Continuous Integration is about. The confusion comes from calling these tools Continuous Integration Services, a better term would be something like "Continuous Build Services". While using a CI Service is a useful aid to doing Continuous Integration, we shouldn't confuse a tool for the practice.

Can a team do both Continuous Integration and Feature Branching at the same time?

In general, Continuous Integration and Feature Branching are mutually exclusive approaches. Most folks who think they are doing both are running a CI Service on their Feature Branches, which as I explain in the previous question, isn't doing Continuous Integration.

There is one situation where it is possible to do both, that is when all the features are so small they can be completed in less than a day. But that seems to be a very rare case, and most people would just call that Continuous Integration.

A secondary point here is that it's perfectly permissible to do personal work on a separate branch, then merge it with main and push when I integrate. I might do that if I were worried I'd fat-finger my IDE and push a broken local main by accident. The key question is whether I'm integrating continuously, not how I manage my personal workspace.

What is the difference between Continuous Integration and Continuous Delivery?

The early descriptions of Continuous Integration focused on the cycle of developer integration with the mainline in the team's development environment. Such descriptions didn't talk much about the journey from an integrated mainline to a production release. That doesn't mean they weren't in people's minds. Practices like “Automate Deployment” and “Test in a Clone of the Production Environment” clearly indicate a recognition of the path to production.

In some situations, there wasn't much else after mainline integration. I recall Kent showing me a system he was working on in Switzerland in the late 90's where they deployed to production, every day, automatically. But this was a Smalltalk system, that didn't have complicated steps for a production deploy. In the early 2000s at Thoughtworks, we often had situations where that path to production was much more complicated. This led to the notion that there was an activity beyond Continuous Integration that addressed that path. That activity came to knows as Continuous Delivery.

The aim of Continuous Delivery is that the product should always be in a state where we can release the latest build. This is essentially ensuring that the release to

production is a business decision.

For many people these days, Continuous Integration is about integrating code to the mainline in the development team's environment, and Continuous Delivery is the rest of the deployment pipeline heading to a production release. Some people treat Continuous Delivery as encompassing Continuous Integration, others see them as closely linked partners, often with the moniker CI/CD. Others argue that Continuous Delivery is merely a synonym for Continuous Integration.

How does Continuous Deployment fit in with all this?

Continuous Integration ensures everyone integrates their code at least daily to the mainline in version control. Continuous Delivery then carries out any steps required to ensure that the product is releasable to product whenever anyone wishes. Continuous Deployment means the product is automatically released to production whenever it passes all the automated tests in the deployment pipeline.

With Continuous Deployment every commit pushed to mainline as part of Continuous Integration will be automatically deployed to production providing all of the verifications in the deployment pipeline are green. Continuous Delivery just assures that this is possible (and is thus a pre-requisite for Continuous Deployment).

How do we do pull requests and code reviews?

Pull Requests, an artifact of GitHub, are now widely used on software projects. Essentially they provide a way to add some process to the push to mainline, usually involving a pre-integration code review, requiring another developer to approve before the push can be accepted into the mainline. They developed mostly in the context of feature branching in open-source projects, ensuring that the maintainers of a project can review that a contribution fits properly into the style and future intentions of the project.

The pre-integration code review can be problematic for Continuous Integration because it usually adds significant friction to the integration process. Instead of an automated process that can be done within minutes, we have to find someone to do the code review, schedule their time, and wait for feedback before the review is accepted. Although some organizations may be able to get to flow within minutes, this

can easily end up being hours or days - breaking the timing that makes Continuous Integration work.

Those who do Continuous Integration deal with this by reframing how code review fits into their workflow. Pair Programming is popular because it creates a continuous real-time code review as the code is being written, producing a much faster feedback loop for the review. The Ship / Show / Ask process encourages teams to use a blocking code review only when necessary, recognizing that post-integration review is often a better bet as it doesn't interfere with integration frequency. Many teams find that Refinement Code Review is an important force to maintaining a healthy code base, but works at its best when Continuous Integration produces an environment friendly to refactoring.

We should remember that pre-integration review grew out of an open-source context where contributions appear impromptu from weakly connected developers. Practices that are effective in that environment need to be reassessed for a full-time team of closely-knit staff.

How do we handle databases?

Databases offer a specific challenge as we increase integration frequency. It's easy to include database schema definitions and load scripts for test data in the version-controlled sources. But that doesn't help us with data outside of version-control, such as production databases. If we change the database schema, we need to know how to handle existing data.

With traditional pre-release integration, data migration is a considerable challenge, often spinning up special teams just to carry out the migration. At first blush, attempting high-frequency integration would introduce an untenable amount of data migration work.

In practice, however, a change in perspective removes this problem. We faced this issue in Thoughtworks on our early projects using Continuous Integration, and solved it by shifting to an Evolutionary Database Design approach, developed by my colleague Pramod Sadalage. The key to this methodology is to define database schema and data through a series of migration scripts, that alter both the database schema and data. Each migration is small, so is easy to reason about and test. The migrations compose naturally, so we can run hundreds of migrations in sequence to perform significant schema changes and migrate the data as we go. We can store these migrations in

version-control in sync with the data access code in the application, allowing us to build any version of the software, with the correct schema and correctly structured data. These migrations can be run on test data, and on production databases.

Final Thoughts

Most software development is about changing existing code. The cost and response time for adding new features to a code base depends greatly upon the condition of that code base. A crufty code base is harder and more expensive to modify. To keep cruft to a minimum a team needs to be able to regularly refactor the code, changing its structure to reflect changing needs and incorporate lessons the team learns from working on the product.

Continuous Integration is vital for a healthy product because it is a key component of this kind of evolutionary design ecosystem. Together with and supported by self-testing code, it's the underpinning for refactoring. These technical practices, born together in Extreme Programming, can enable a team to deliver regular enhancement of a product to take advantage of changing needs and technological opportunities.

Further Reading

An essay like this can only cover so much ground, but this is an important topic so I've created a guide page on my website to point you to more information.

To explore Continuous Integration in more detail I suggest taking a look at Paul Duvall's appropriately titled book on the subject (which won a Jolt award - more than I've ever managed). For more on the broader process of Continuous Delivery, take a look at Jez Humble and Dave Farley's book - which also beat me to a Jolt award.

My article on Patterns for Managing Source Code Branches looks at the broader context, showing how Continuous Integration fits into the wider decision space of choosing a branching strategy. As ever, the driver for choosing when to branch is knowing you are going to integrate.

The original article on Continuous Integration describes our experiences as Matt helped put together continuous integration on a Thoughtworks project in 2000.

As I indicated earlier, many people write about Continuous Integration using the term “Trunk-Based Development”. Paul Hammant's website contains a lot of useful and practical information. Clare Sudbery recently wrote an informative report available through O'Reilly.

Acknowledgments

First and foremost to Kent Beck and my many colleagues on the Chrysler Comprehensive Compensation (C3) project. This was my first chance to see Continuous Integration in action with a meaningful amount of unit tests. It showed me what was possible and gave me an inspiration that led me for many years.

Thanks to Matt Foemmel, Dave Rice, and everyone else who built and maintained Continuous Integration on Atlas. That project was a sign of CI on a larger scale and showed the benefits it made to an existing project.

Paul Julius, Jason Yip, Owen Rodgers, Mike Roberts and many other open source contributors have participated in building some variant of CruiseControl, the first CI service. Although a CI service isn't essential, most teams find it helpful. CruiseControl and other CI services have played a big part in popularizing and enabling software developers to use Continuous Integration.

In the fall of 2023, Michael Lihs emailed me with suggested revisions to the article, which inspired me to do a major overhaul. Birgitta Böckeler, Camilla Crispim, Casey Lee, Chris Ford, Clare Sudbery, Evan Bottcher, Jez Humble, Kent Beck, Kief Morris, Mike Roberts, Paul Hammant, Pete Hodgson, Rafael Detoni, Rouan Wilsenach, and Trisha Gee reviewed and commented on this revision.

One of the reasons I work at Thoughtworks is to get good access to practical projects done by talented people. Nearly every project I've visited has given tasty morsels of continuous integration information.

▶ Significant Revisions

© Martin Fowler | Disclosures